
TERRAFORMING THE WELL-ARCHITECTED WAY

IaC Reviews That Actually Catch Risks

Alam Zaib Ahmed

Managed Services Support Analyst — NOC @ boxxe

AWS Community Builder Dev-ops-Tools

AWS MEETUP · LONDON · JUN

OVERVIEW

AGENDA

01

The IaC Risk Landscape

Why most Terraform reviews miss what matters

02

The Well-Architected Review Framework

Mapping the six pillars to your Terraform workflow

03

Building Your Review Pipeline

Tools, metrics, and culture for 2026

01

THE IaC RISK LANDSCAPE

Why most Terraform reviews miss what matters

Why Most IaC Reviews Still Fail

80%

of cloud security incidents stem from misconfigurations

- | Manual reviews catch syntax errors but miss security-critical patterns
- | Open security groups, unencrypted storage, overprivileged IAM, public S3 buckets
- | The "LGTM" problem: rubber-stamp approvals under delivery pressure
- | Reviews treated as gatekeeping, not as risk detection conversations

A black and white line drawing of the London skyline. From left to right, it features the Gheron (a curved, ribbed structure), the London Eye (a large Ferris wheel), and Big Ben (a tall clock tower). The background is a simple line drawing of the city skyline with various buildings and a bridge. The drawing is done in a sketchy, hand-drawn style.

Team Maturity



- ✓ **Implicit Dependency Chains**
 - Implicit dependency chains
 - Hardcoded IDs creating breaking changes
- ✓ **State File Sprawl**
 - State File Sprawl Groups
 - ✗ Secrets-Corrections
- ✓ **Secrets in .tfvars**
 - Credentials committed to VC despite intentions
- ✓ **0.0.0.0/0 Security Groups**
 - 0.0.0.0/0 Security Groups
 - ✗ Defaults Encryptions
- ✓ **Default Encryption Keys**
 - Default encryption
 - Encryption Keys ✗
- ✓ **Missing Cost Tags**
 - Ensoete Encryption ✗
 - Missing Cost Tags ✓ ✗
- ✓ **Wildcard IAM Policies**
 - Wildcard IAM Policies ✗

The Real Cost of a Missed Risk

PRE-DEPLOY

Minutes

To fix a misconfiguration caught in development

POST-DEPLOY

Days

Potential downtime, data exposure, incident response

-
- | **44%** of organisations experience environment drift from manual changes
 - | Blast radius of a bad Terraform apply: state corruption, cascading failures
 - | Shift-left is not enough — you need continuous validation at every layer

02

THE WELL-ARCHITECTED FRAMEWORK

Mapping the six pillars to your Terraform workflow

From Checklists to Codified Rules

Well-Architected is no longer a once-a-year exercise. It is a living operating model.

PILLAR	TERRAFORM CHECK
Operational Excellence	Resource tagging, CloudWatch logging, traceability
Security	Encryption at rest/transit, least privilege IAM, private subnets
Reliability	Multi-AZ deployment, automated backups, health checks
Performance Efficiency	Right-sized instances, auto-scaling, caching strategies
Cost Optimisation	FinOps tagging, Reserved Instance analysis, lifecycle policies
Sustainability	Carbon-aware region selection, Graviton migration, idle detection

Domain lenses in 2026: GenAI, IoT, Streaming — each with Terraform-specific gotchas

The Four Review Layers

01 IDE

Real-time feedback as you write — tflint, Checkov plugins

02 Pull Request

Mandatory scanning on every PR — terraform plan + security scan

03 CI/CD Pipeline

Policy gates block deployments with critical findings

04 Runtime

Continuous drift detection and misconfiguration monitoring

Each layer catches what the previous one missed. Defence in depth.

03

BUILDING YOUR REVIEW PIPELINE

Tools, metrics, and culture for 2026

Policy as Code: Enforcement, Not Documentation

DOCUMENTATION

"Storage buckets should not be public" — written in a wiki nobody reads



ENFORCEMENT

A rule that blocks public buckets entirely — enforced on every pull request

- | OPA/Rego, Checkov, Sentinel — codify once, enforce everywhere
- | Start with CIS AWS Foundations Benchmark as your baseline
- | Policy-as-code lives in version control alongside your Terraform modules

Your 2026 Tooling Stack

CATEGORY	TOOLS	WHEN TO RUN
Static Analysis	Checkov, Trivy, TFLint	On every save and PR
Policy Enforcement	OPA, Sentinel	At CI gate, blocks deploy
Cost Visibility	Infracost	In PR, before merge
Drift Detection	Terraform Cloud, AWS Config	Continuously, scheduled
Secrets Management	Vault, AWS Secrets Manager	Runtime, never in code
Testing	Terratest, terraform validate	Pre-deploy, in CI

Measuring What Matters

MTTD

Mean Time to Detect

How fast you catch a misconfiguration

MTTR

Mean Time to Remediate

From detection to fixed in production

-
- | **Policy Coverage** — percentage of Well-Architected questions codified
 - | **Drift Rate** — percentage of resources diverged from IaC definitions
 - | **False Positive Rate** — if above 20%, developers bypass your gates

Review these metrics monthly, not quarterly. Trend them, do not just snapshot them.

START WITH ONE PILLAR

Pick your riskiest pillar. Codify three checks. Run them on every PR.

Build from there.

THANK YOU

Questions?



Link to AWS Community builders



LinkedIn: www.linkedin.com/in/alam-ahmed-cloud-engineer